

Recommendation System Project: IBM Community

In this notebook, you will be putting your recommendation skills to use on real data from the IBM Watson Studio platform.

You may either submit your notebook through the workspace here, or you may work from your local machine and submit through the next page. Either way assure that your code passes the project [RUBRIC](#). **Please save regularly.**

By following the table of contents, you will build out a number of different methods for making recommendations that can be used for different situations.

Table of Contents

- I. [Exploratory Data Analysis](#)
- II. [Rank Based Recommendations](#)
- III. [User-User Based Collaborative Filtering](#)
- IV. [Content Based Recommendations](#)
- V. [Matrix Factorization](#)
- VI. [Extras & Concluding](#)

At the end of the notebook, you will find directions for how to submit your work. Let's get started by importing the necessary libraries and reading in the data.

```
In [1]: %pip install matplotlib
```

```
Requirement already satisfied: matplotlib in c:\python312\lib\site-packages (3.1
0.9)
Requirement already satisfied: contourpy>=1.0.1 in c:\python312\lib\site-packages
(from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in c:\python312\lib\site-packages (fr
om matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in c:\python312\lib\site-package
s (from matplotlib) (4.62.1)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\python312\lib\site-package
s (from matplotlib) (1.5.0)
Requirement already satisfied: numpy>=1.23 in c:\python312\lib\site-packages (fro
m matplotlib) (2.4.4)
Requirement already satisfied: packaging>=20.0 in c:\users\rambo406\appdata\roami
ng\python\python312\site-packages (from matplotlib) (26.2)
Requirement already satisfied: pillow>=8 in c:\python312\lib\site-packages (from
matplotlib) (12.2.0)
Requirement already satisfied: pyparsing>=3 in c:\python312\lib\site-packages (fr
om matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\rambo406\appdata
\roaming\python\python312\site-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in c:\users\rambo406\appdata\roaming\pyth
on\python312\site-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
Note: you may need to restart the kernel to use updated packages.
```

```
[notice] A new release of pip is available: 25.3 -> 26.1.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

```
In [2]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import project_tests as t

df = pd.read_csv(
    'data/user-item-interactions.csv'
)
df['article_id'] = df['article_id'].astype(int)
# Show df to get an idea of the data
df.head()
```

```
Out[2]:
```

	Unnamed: 0	article_id	title	email
0	0	1430	using pixiedust for fast, flexible, and easier...	ef5f11f77ba020cd36e1105a00ab868bbdbf7fe7
1	1	1314	healthcare python streaming application demo	083cbdfa93c8444beaa4c5f5e0f5f9198e4f9e0b
2	2	1429	use deep learning for image classification	b96a4f2e92d8572034b1e9b28f9ac673765cd074
3	3	1338	ml optimization using cognitive assistant	06485706b34a5c9bf2a0ecdac41daf7e7654ceb7
4	4	1276	deploy your python model as a restful api	f01220c46fc92c6e6b161b1849de11faacd7ccb2

Part I : Exploratory Data Analysis

Use the dictionary and cells below to provide some insight into the descriptive statistics of the data.

1. Are there any missing values? If so, provide a count of missing values. If there are missing values in `email`, assign it the same id value `"unknown_user"`.

```
In [3]: # Some interactions do not have a user associated with it, assume the same user.
df.info()
```

```

<class 'pandas.DataFrame'>
RangeIndex: 45993 entries, 0 to 45992
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   Unnamed: 0   45993 non-null  int64
1   article_id   45993 non-null  int64
2   title        45993 non-null  str
3   email        45976 non-null  str
dtypes: int64(2), str(2)
memory usage: 1.4 MB

```

```
In [4]: print(f"Number of Null email values is: {df['email'].isna().sum()}")
```

Number of Null email values is: 17

```
In [5]: df[df.email.isna()]
```

```
Out[5]:
```

	Unnamed: 0	article_id	title	email
25131	25146	1016	why you should master r (even if it might even...	NaN
29758	30157	1393	the nurse assignment problem	NaN
29759	30158	20	working interactively with rstudio and noteboo...	NaN
29760	30159	1174	breast cancer wisconsin (diagnostic) data set	NaN
29761	30160	62	data visualization: the importance of excludin...	NaN
35264	36016	224	using apply, sapply, lapply in r	NaN
35276	36029	961	beyond parallelize and collect	NaN
35277	36030	268	sector correlations shiny app	NaN
35278	36031	268	sector correlations shiny app	NaN
35279	36032	268	sector correlations shiny app	NaN
35280	36033	268	sector correlations shiny app	NaN
35281	36034	415	using machine learning to predict value of hom...	NaN
35282	36035	846	pearson correlation aggregation on sparksql	NaN
35283	36036	268	sector correlations shiny app	NaN
35284	36037	162	an introduction to stock market data analysis ...	NaN
42749	44112	647	getting started with apache mahout	NaN
42750	44113	965	data visualization playbook: revisiting the ba...	NaN

```
In [6]: # Fill email NaNs with "unknown_user"
df['email'] = df['email'].fillna('unknown_user')
```

```
In [7]: # Check if no more NaNs
df[df.email.isna()]
```

```
Out[7]:
```

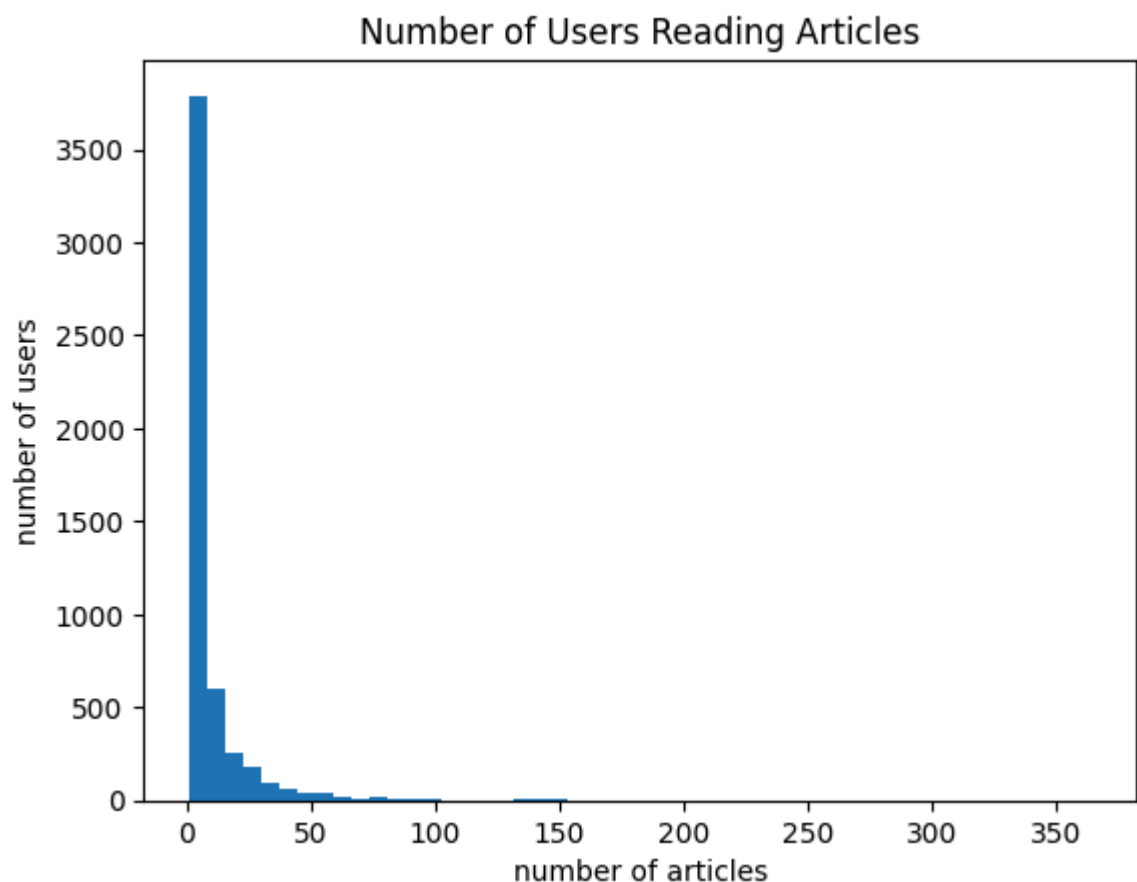
	Unnamed: 0	article_id	title	email
--	------------	------------	-------	-------

2. What is the distribution of how many articles a user interacts with in the dataset? Provide a visual and descriptive statistics to assist with giving a look at the number of times each user interacts with an article.

```
In [8]: # What are the descriptive statistics of the number of articles a user interacts with
user_interactions = df.groupby('email').size()
user_interactions.describe()
```

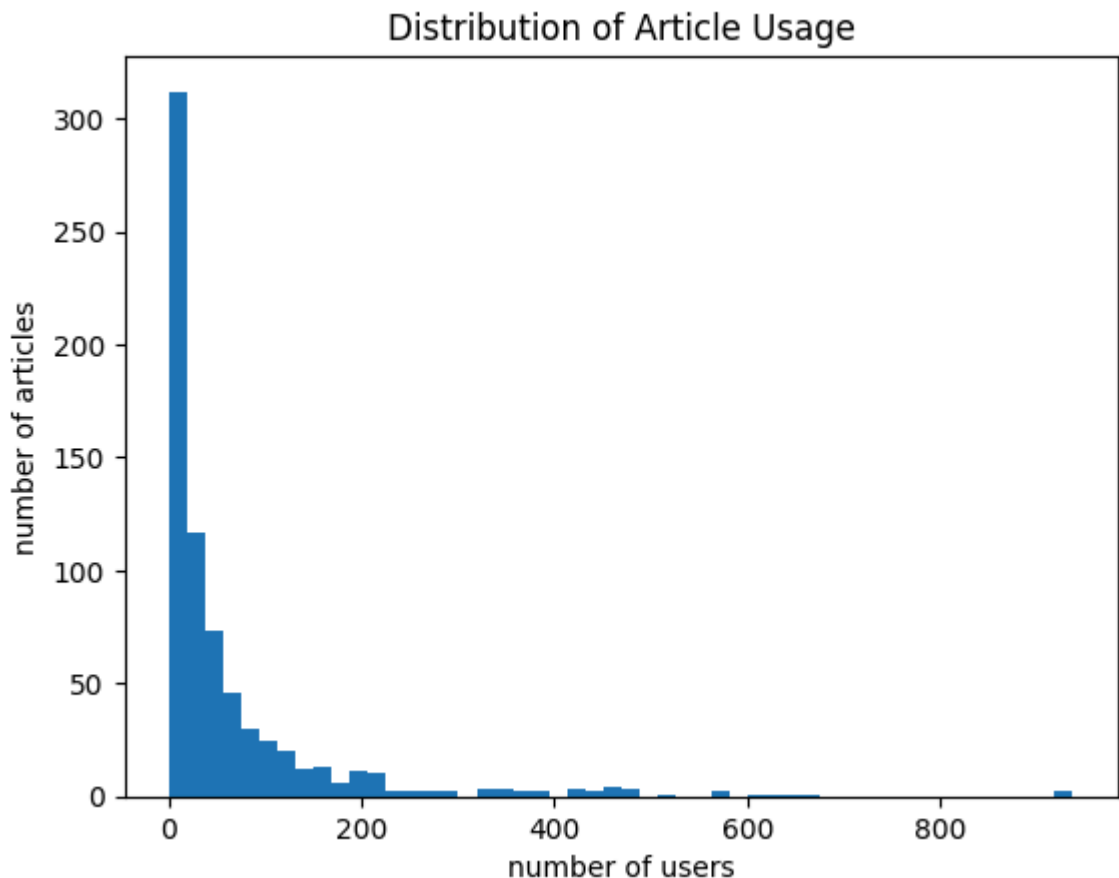
```
Out[8]: count    5149.000000
mean         8.932414
std         16.801011
min          1.000000
25%          1.000000
50%          3.000000
75%          9.000000
max        364.000000
dtype: float64
```

```
In [9]: # Create a plot of the number of articles read by each user
user_interactions = df.groupby('email').size()
plt.hist(user_interactions, bins=50)
plt.xlabel('number of articles')
plt.ylabel('number of users')
plt.title('Number of Users Reading Articles')
plt.show()
```



```
In [10]: # Create a plot of the number of times each article was read
article_interactions = df.groupby('article_id').size()
plt.hist(article_interactions, bins=50)
plt.xlabel('number of users')
plt.ylabel('number of articles')
```

```
plt.title('Distribution of Article Usage')
plt.show()
```



```
In [13]: most_viewed_article_id = 1429 # The most viewed article in the dataset as a str
max_views = 937 # The most viewed article in the dataset was viewed how many ti
```

```
In [14]: ## No need to change the code here - this will be helpful for later parts of the
# Run this cell to map the user email to a user_id column and remove the email c

def email_mapper(df=df):
    """Map unique user emails to sequential integer user IDs.

    INPUT:
    df - (pandas dataframe) dataframe with an 'email' column

    OUTPUT:
    user_ids - (list) a list of integer user IDs corresponding to each row's ema
    """
    coded_dict = {
        email: num
        for num, email in enumerate(df['email'].unique(), start=1)
    }
    return [coded_dict[val] for val in df['email']]

df['user_id'] = email_mapper(df)
del df['email']

# show header
df.head()
```

```
Out[14]:
```

	Unnamed: 0	article_id	title	user_id
0	0	1430	using pixiedust for fast, flexible, and easier...	1
1	1	1314	healthcare python streaming application demo	2
2	2	1429	use deep learning for image classification	3
3	3	1338	ml optimization using cognitive assistant	4
4	4	1276	deploy your python model as a restful api	5

```
In [15]: ## If you stored all your results in the variable names above,
## you shouldn't need to change anything in this cell

sol_1_dict = {
    '50% of individuals have ____ or fewer interactions.': median_val,
    'The total number of user-article interactions in the dataset is ____.':
    'The maximum number of user-article interactions by any 1 user is ____.':
    'The most viewed article in the dataset was viewed ____ times.': max_view
    'The article_id of the most viewed article is ____.': most_viewed_articl
    'The number of unique articles that have at least 1 rating ____.': uniqu
    'The number of unique users in the dataset is ____.': unique_users,
    'The number of unique articles on the IBM platform': total_articles
}

# Test your dictionary against the solution
t.sol_1_test(sol_1_dict)
```

It looks like you have everything right here! Nice job!

Part II: Rank-Based Recommendations

In this project, we don't actually have ratings for whether a user liked an article or not. We only know that a user has interacted with an article. In these cases, the popularity of an article can really only be based on how often an article was interacted with.

1. Fill in the function below to return the **n** top articles ordered with most interactions as the top. Test your function using the tests below.

```
In [16]: def get_top_articles(n, df=df):
        """
        INPUT:
        n - (int) the number of top articles to return
        df - (pandas dataframe) df as defined at the top of the notebook

        OUTPUT:
        top_articles - (list) A list of the top 'n' article titles

        """
        top_articles = df.groupby(['title', 'article_id']).size().reset_index(name='')

        return top_articles # Return the top article titles from df

def get_top_article_ids(n, df=df):
    """
    INPUT:
    n - (int) the number of top articles to return
    df - (pandas dataframe) df as defined at the top of the notebook

    OUTPUT:
    top_articles - (list) A list of the top 'n' article titles

    """
    top_articles = df['article_id'].value_counts().head(n).index.tolist()

    return top_articles # Return the top article ids
```

```
In [17]: print(get_top_articles(10))
        print(get_top_article_ids(10))
```

```
['use deep learning for image classification', 'insights from new york car accide
nt reports', 'visualize car data with brunel', 'use xgboost, scikit-learn & ibm w
atson machine learning apis', 'predicting churn with the spss random tree algorit
hm', 'healthcare python streaming application demo', 'finding optimal locations o
f new store using decision optimization', 'apache spark lab, part 1: basic concep
ts', 'analyze energy consumption in buildings', 'gosales transactions for logisti
c regression model']
[1429, 1330, 1431, 1427, 1364, 1314, 1293, 1170, 1162, 1304]
```

```
In [18]: # Test your function by returning the top 5, 10, and 20 articles
        top_5 = get_top_articles(5)
        top_10 = get_top_articles(10)
        top_20 = get_top_articles(20)

        # Test each of your three lists from above
        t.sol_2_test(get_top_articles)
```

Your top_5 looks like the solution list! Nice job.
Your top_10 looks like the solution list! Nice job.
Your top_20 looks like the solution list! Nice job.

Part III: User-User Based Collaborative Filtering

1. Use the function below to reformat the **df** dataframe to be shaped with users as the rows and articles as the columns.

- Each **user** should only appear in each **row** once.
- Each **article** should only show up in one **column**.
- **If a user has interacted with an article, then place a 1 where the user-row meets for that article-column.** It does not matter how many times a user has interacted with the article, all entries where a user has interacted with an article should be a 1.
- **If a user has not interacted with an item, then place a zero where the user-row meets for that article-column.**

Use the tests to make sure the basic structure of your matrix matches what is expected by the solution.

```
In [19]: # create the user-article matrix with 1's and 0's

def create_user_item_matrix(df, fill_value=0):
    """
    INPUT:
    df - pandas dataframe with article_id, title, user_id columns

    OUTPUT:
    user_item - user item matrix

    Description:
    Return a matrix with user ids as rows and article ids on the columns with 1
    an article and a 0 otherwise
    """
    user_item = df.groupby(['user_id', 'article_id']).size().unstack(fill_value=
    user_item = (user_item > 0).astype(int)

    return user_item # return the user_item matrix

user_item = create_user_item_matrix(df)
```

```
In [20]: ## Tests: You should just need to run this cell. Don't change the code.
assert user_item.shape[0] == 5149, "Oops! The number of users in the user-artic
assert user_item.shape[1] == 714, "Oops! The number of articles in the user-art
assert user_item.sum(axis=1)[1] == 36, "Oops! The number of articles seen by us
print("You have passed our quick tests! Please proceed!")
```

You have passed our quick tests! Please proceed!

2. Complete the function below which should take a `user_id` and provide an ordered list of the most similar users to that user (from most similar to least similar). The returned result should not contain the provided `user_id`, as we know that each user is similar to him/herself. Because the results for each user here are binary, it (perhaps) makes sense to compute similarity as the dot product of two users.

Use the tests to test your function.


```
In [21]: # Lets use the cosine_similarity function from sklearn
from sklearn.metrics.pairwise import cosine_similarity
```

```
In [22]: def find_similar_users(user_id, user_item=user_item, include_similarity=False):
        """
        INPUT:
        user_id - (int) a user_id
        user_item - (pandas dataframe) matrix of users by articles:
                     1's when a user has interacted with an article, 0 otherwise
        include_similarity - (bool) whether to include the similarity in the output

        OUTPUT:
        similar_users - (list) an ordered list where the closest users (largest dot
                          are listed first

        Description:
        Computes the similarity of every pair of users based on the dot product
        Returns an ordered list of user ids. If include_similarity is True, returns
        where the first element is the user id and the second the similarity.

        """
        # compute similarity of each user to the provided user
        user_row = user_item.loc[user_id].values.reshape(1, -1)
        similarities = cosine_similarity(user_row, user_item.values).flatten()

        # sort by similarity
        sim_df = pd.DataFrame({
            'user_id': user_item.index,
            'similarity': similarities
        })
        # remove the own user's id
        sim_df = sim_df[sim_df['user_id'] != user_id]
        sim_df = sim_df.sort_values('similarity', ascending=False)

        # create list of just the ids
        most_similar_users = sim_df['user_id'].tolist()

        if include_similarity:
            return sim_df[['user_id', 'similarity']].values.tolist()
        return most_similar_users # return a list of the users in order from most to
```

```
In [23]: # Do a spot check of your function
print("The 10 most similar users to user 1 are: {}".format(find_similar_users(1)
print("The 5 most similar users to user 3933 are: {}".format(find_similar_users(
print("The 3 most similar users to user 46 are: {}".format(find_similar_users(46
```

The 10 most similar users to user 1 are: [3933, 4201, 46, 824, 5034, 253, 5041, 136, 2305, 395]

The 5 most similar users to user 3933 are: [1, 46, 4201, 824, 5034]

The 3 most similar users to user 46 are: [4201, 790, 5077]

3. Now that you have a function that provides the most similar users to each user, you will want to use these users to find articles you can recommend. Complete the functions below to return the articles you would recommend to each user.

```
In [24]: def get_article_names(article_ids, df=df):
        """
        INPUT:
```

article_ids - (list) a list of article ids
df - (pandas dataframe) df as defined at the top of the notebook

OUTPUT:

article_names - (list) a list of article names associated with the list of article_ids
(this is identified by the title column in df)

"""

```
article_names = []
for aid in article_ids:
    titles = df[df['article_id'] == aid]['title'].unique()
    if len(titles) > 0:
        article_names.append(titles[0])
```

return article_names *# Return the article names associated with list of article_ids*

```
def get_ranked_article_unique_counts(article_ids, user_item=user_item):
```

"""

INPUT:

article_ids - (list) a list of article ids to evaluate (a subset to rank).
the dtype should match the column labels in `user\_item`.

user_item - (pandas dataframe) a binary matrix of users by articles [n_users x n_articles]
1's when a user has interacted with an article, 0 otherwise
columns are article IDs.

OUTPUT:

ranked_article_unique_counts - (list) a list of tuples [(article_id, unique_users)]
only for the provided `article\_ids` with article_id and number of unique users that have interacted with the article, sorted by the number of unique users in descending order

Description:

For each `article\_id` in the input list, count the number of unique users who have interacted with it (column sum in the binary `user\_item` matrix).
Return the (article_id, count) pairs sorted in descending order.

"""

```
counts = []
for aid in article_ids:
    if aid in user_item.columns:
        count = user_item[aid].sum()
        counts.append([aid, count])
    else:
        counts.append([aid, 0])
counts.sort(key=lambda x: x[1], reverse=True)
```

return counts

```
def get_user_articles(user_id, user_item=user_item):
```

"""

INPUT:

user_id - (int) a user id

user_item - (pandas dataframe) matrix of users by articles:
1's when a user has interacted with an article, 0 otherwise

OUTPUT:

article_ids - (list) a list of the article ids seen by the user

article_names - (list) a list of article names associated with the list of article_ids
(this is identified by the title column in df)

Description:

```

    Provides a list of the article_ids and article titles that have been seen by
    """
    article_ids = user_item.loc[user_id][user_item.loc[user_id] == 1].index.tolist()
    article_names = get_article_names(article_ids)

    return article_ids, article_names # return the ids and names

def user_user_recs(user_id, m=10):
    """
    INPUT:
    user_id - (int) a user id
    m - (int) the number of recommendations you want for the user

    OUTPUT:
    recs - (list) a list of recommendations for the user

    Description:
    Loops through the users based on closeness to the input user_id
    For each user - finds articles the user hasn't seen before and provides them
    Does this until m recommendations are found

    Notes:
    Users who are the same closeness are chosen arbitrarily as the 'next' user

    For the user where the number of recommended articles starts below m
    and ends exceeding m, the last items are chosen arbitrarily

    """
    seen_articles, _ = get_user_articles(user_id)
    similar_users = find_similar_users(user_id)

    recs = []
    for sim_user in similar_users:
        sim_user_articles, _ = get_user_articles(sim_user)
        new_articles = [a for a in sim_user_articles if a not in seen_articles]
        recs.extend(new_articles)
        if len(recs) >= m:
            break

    return recs[:m] # return your recommendations for this user_id

```

```

In [25]: # Check Results
get_article_names(user_user_recs(1, 10)) # Return 10 recommendations for user 1

```

```

Out[25]: ['this week in data science (april 18, 2017)',
          'this week in data science (may 2, 2017)',
          'top 20 r machine learning and data science packages',
          'improving the roi of big data and analytics through leveraging new sources of
          data',
          'using apply, sapply, lapply in r',
          'awesome deep learning papers',
          'leverage python, scikit, and text classification for behavioral profiling',
          'challenges in deep learning',
          'do i need to learn r?',
          'how can data scientists collaborate to build better business']

```

```

In [26]: get_ranked_article_unique_counts([1320, 232, 844])

```

```
Out[26]: [[1320, np.int64(123)], [844, np.int64(78)], [232, np.int64(62)]]
```

```
In [27]: # Test your functions here - No need to change this code - just run this cell
assert set(get_article_names([1024, 1176, 1305, 1314, 1422, 1427])) == set(['usi
assert set(get_article_names([1320, 232, 844])) == set(['housing (2015): united
assert set(get_user_articles(20)[0]) == set([1320, 232, 844])
assert set(get_user_articles(20)[1]) == set(['housing (2015): united states demo
assert set(get_user_articles(2)[0]) == set([1024, 1176, 1305, 1314, 1422, 1427])
assert set(get_user_articles(2)[1]) == set(['using deep learning to reconstruct
assert get_ranked_article_unique_counts([1320, 232, 844])[0] == [1320, 123], "Oo
print("If this is all you see, you passed all of our tests! Nice job!")
```

If this is all you see, you passed all of our tests! Nice job!

4. Now we are going to improve the consistency of the **user_user_recs** function from above.

- Instead of arbitrarily choosing when we obtain users who are all the same closeness to a given user - choose the users that have the most total article interactions before choosing those with fewer article interactions.
- Instead of arbitrarily choosing articles from the user where the number of recommended articles starts below m and ends exceeding m, choose articles with the articles with the most total interactions before choosing those with fewer total interactions. This ranking should be what would be obtained from the **top_articles** function you wrote earlier.

```
In [28]: def get_top_sorted_users(user_id, user_item=user_item):
        """
        INPUT:
        user_id - (int)
        user_item - (pandas dataframe) matrix of users by articles:
                    1's when a user has interacted with an article, 0 otherwise

        OUTPUT:
        neighbors_df - (pandas dataframe) a dataframe with:
                        neighbor_id - is a neighbor user_id
                        similarity - measure of the similarity of each user to the p
                        num_interactions - the number of articles viewed by the user

        Other Details - sort the neighbors_df by the similarity and then by number o
                        highest of each is higher in the dataframe, i.e. Descending

        """
        similar_with_sim = find_similar_users(user_id, user_item=user_item, include_

        neighbors_df = pd.DataFrame(similar_with_sim, columns=['neighbor_id', 'simil
        neighbors_df['neighbor_id'] = neighbors_df['neighbor_id'].astype(int)
        neighbors_df['num_interactions'] = neighbors_df['neighbor_id'].apply(
            lambda x: user_item.loc[x].sum()
        )
        neighbors_df = neighbors_df.sort_values(['similarity', 'num_interactions'],
        neighbors_df = neighbors_df.reset_index(drop=True)

        return neighbors_df # Return the dataframe specified in the doc_string
```

```

def user_user_recs_part2(user_id, m=10):
    """
    INPUT:
    user_id - (int) a user id
    m - (int) the number of recommendations you want for the user

    OUTPUT:
    recs - (list) a list of recommendations for the user by article id
    rec_names - (list) a list of recommendations for the user by article title

    Description:
    Loops through the users based on closeness to the input user_id
    For each user - finds articles the user hasn't seen before and provides them
    Does this until m recommendations are found

    Notes:
    * Choose the users that have the most total article interactions
    before choosing those with fewer article interactions.

    * Choose articles with the articles with the most total interactions
    before choosing those with fewer total interactions.

    """
    seen_articles, _ = get_user_articles(user_id)
    neighbors_df = get_top_sorted_users(user_id)

    recs = []
    for _, row in neighbors_df.iterrows():
        sim_user = int(row['neighbor_id'])
        sim_user_articles, _ = get_user_articles(sim_user)
        new_articles = [a for a in sim_user_articles if a not in seen_articles]
        if new_articles:
            ranked = get_ranked_article_unique_counts(new_articles)
            new_articles_sorted = [a[0] for a in ranked]
            recs.extend(new_articles_sorted)
        if len(recs) >= m:
            break

    return recs[:m], get_article_names(recs[:m]) # return your recommendations f

```

```

In [29]: # Quick spot check - don't change this code - just use it to test your functions
rec_ids, rec_names = user_user_recs_part2(20, 10)
print("The top 10 recommendations for user 20 are the following article ids:")
print(rec_ids)
print()
print("The top 10 recommendations for user 20 are the following article names:")
print(rec_names)

```

The top 10 recommendations for user 20 are the following article ids:
[1185, 1162, 1165, 1293, 254, 1271, 1328, 1410, 40, 1402]

The top 10 recommendations for user 20 are the following article names:
['classify tumors with machine learning', 'analyze energy consumption in building
s', 'analyze precipitation data', 'finding optimal locations of new store using d
ecision optimization', 'apple, ibm add machine learning to partnership with watso
n-core ml coupling', 'customer demographics and sales', 'income (2015): united st
ates demographic measures', 'uci: sms spam collection', 'ensemble learning to imp
rove machine learning results', 'uci: adult - predict income']

5. Use your functions from above to correctly fill in the solutions to the dictionary below. Then test your dictionary against the solution. Provide the code you need to answer each following the comments below.

```
In [30]: print(get_top_sorted_users(1, user_item=user_item).head(n=1))
print(get_top_sorted_users(2, user_item=user_item).head(n=10))
print(get_top_sorted_users(131, user_item=user_item).head(n=10))
```

	neighbor_id	similarity	num_interactions
0	3933	0.986013	35

	neighbor_id	similarity	num_interactions
0	5083	0.730297	5
1	1552	0.577350	2
2	1890	0.577350	2
3	1372	0.471405	3
4	2941	0.433013	8
5	3586	0.408248	4
6	2360	0.408248	1
7	2234	0.408248	1
8	4897	0.408248	1
9	861	0.408248	1

	neighbor_id	similarity	num_interactions
0	3870	0.986667	75
1	4459	0.388909	96
2	203	0.388909	96
3	3782	0.387585	135
4	4932	0.384308	52
5	40	0.384308	52
6	23	0.377647	135
7	242	0.375823	59
8	3910	0.372678	60
9	383	0.367423	32

```
In [31]: ### Tests with a dictionary of results
user1_most_sim = 3933 # Find the user that is most similar to user 1
user2_6th_sim = 3586 # Find the 6th most similar user to user 2
user131_10th_sim = 383 # Find the 10th most similar user to user 131
```

```
In [32]: ## Dictionary Test Here
sol_5_dict = {
    'The user that is most similar to user 1.': user1_most_sim,
    'The user that is the 6th most similar to user 2.': user2_6th_sim,
    'The user that is the 10th most similar to user 131.': user131_10th_sim,
}

t.sol_5_test(sol_5_dict)
```

This all looks good! Nice job!

6. If we were given a new user, which of the above functions would you be able to use to make recommendations? Explain. Can you think of a better way we might make recommendations? Use the cell below to explain a better method for new users.

For a new user with no interaction history, we cannot use user-user collaborative filtering since there are no past interactions to find similar users. Instead, we would use **rank-based recommendations** — recommending the most popular articles overall. This is a reasonable fallback because popular articles are likely to be relevant to a broad audience.

A better approach could combine content-based filtering (if we know user interests/profile) with popularity-based recommendations, or use a hybrid approach that transitions from popularity-based to collaborative filtering as the user builds interaction history.

7. Using your existing functions, provide the top 10 recommended articles you would provide for the a new user below. You can test your function against our thoughts to make sure we are all on the same page with how we might make a recommendation.

```
In [33]: # What would your recommendations be for this new user 0? As a new user, they h
# Provide a list of the top 10 article ids you would give to
new_user_recs = get_top_article_ids(10)
```

```
In [34]: assert set(new_user_recs) == {1314, 1429, 1293, 1427, 1162, 1364, 1304, 1170, 14
print("That's right! Nice job!")
```

That's right! Nice job!

Part IV: Content Based Recommendations

Another method we might use to make recommendations is to recommend similar articles that are possibly related. One way we can find article relationships is by clustering text about those articles. Let's consider content to be the article **title**, as it is the only text we have available. One point to highlight, there isn't one way to create a content based recommendation, especially considering that text information can be processed in many ways.

1. Use the function bodies below to create a content based recommender function `make_content_recs`. We'll use TF-IDF to create a matrix based off article titles, and use this matrix to create clusters of related articles. You can use this function to make recommendations of new articles.

```
In [35]: df.head()
```

```
Out[35]:
```

	Unnamed: 0	article_id	title	user_id
0	0	1430	using pixiedust for fast, flexible, and easier...	1
1	1	1314	healthcare python streaming application demo	2
2	2	1429	use deep learning for image classification	3
3	3	1338	ml optimization using cognitive assistant	4
4	4	1276	deploy your python model as a restful api	5

```
In [36]: from sklearn.cluster import KMeans
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import Normalizer
from sklearn.decomposition import TruncatedSVD
```

```
In [37]: # unique articles
df_unique_articles = df[['article_id', 'title']].drop_duplicates(subset='article
```

```
In [38]: # Create a vectorizer using TfidfVectorizer and fit it to the article titles
max_features = 200
max_df = 0.75
min_df = 5

vectorizer = TfidfVectorizer(
    max_df=max_df,
    min_df=min_df,
    stop_words="english",
    max_features=max_features,
)
print("Running TF-IDF")
X_tfidf = vectorizer.fit_transform(df_unique_articles['title'])

print(f"n_samples: {X_tfidf.shape[0]}, n_features: {X_tfidf.shape[1]}")

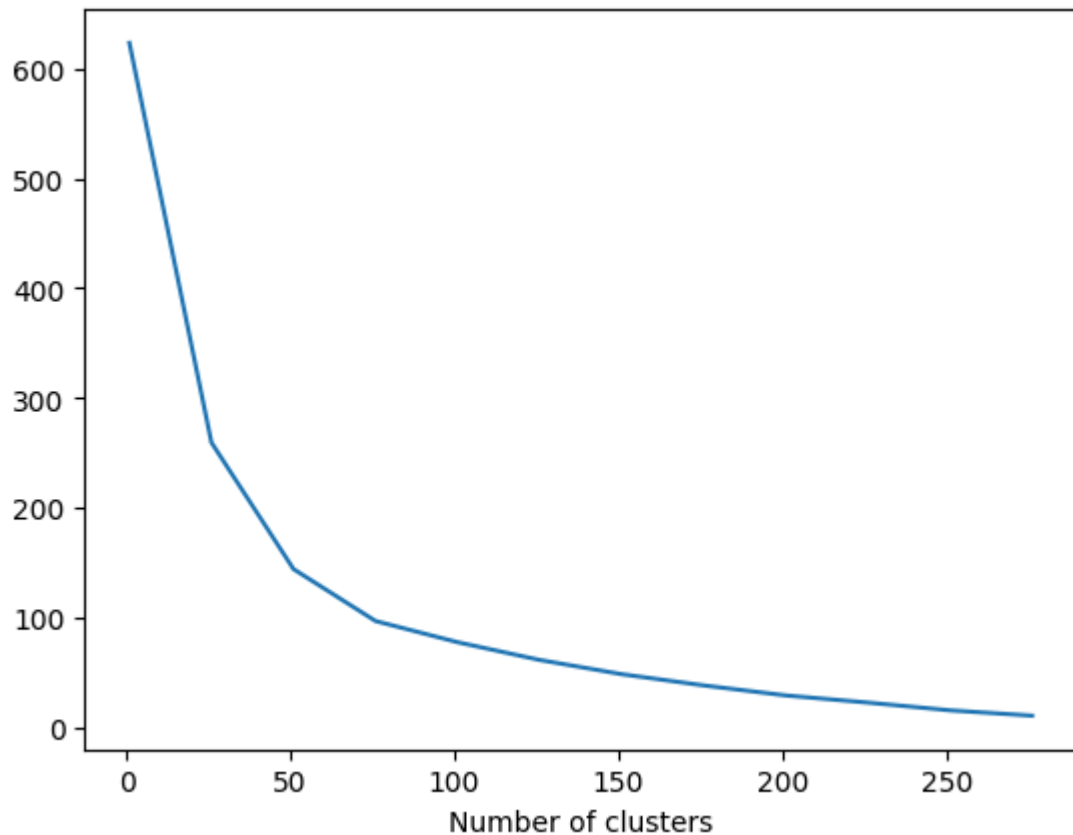
lsa = make_pipeline(TruncatedSVD(n_components=50), Normalizer(copy=False))
X_lsa = lsa.fit_transform(X_tfidf)
explained_variance = lsa[0].explained_variance_ratio_.sum()

print(f"Explained variance of the SVD step: {explained_variance * 100:.1f}%")
```

Running TF-IDF
n_samples: 714, n_features: 125
Explained variance of the SVD step: 76.0%

```
In [39]: # Let's map the inertia for different number of clusters to find the optimal num
# We'll plot it to see the elbow
inertia = []
clusters = 300
step = 25
max_iter = 50
n_init = 5
random_state = 42
for k in range(1, clusters, step):
    kmeans = KMeans(
        n_clusters=k,
        max_iter=max_iter,
        n_init=n_init,
        random_state=random_state,
    ).fit(X_lsa)
    # inertia is the sum of squared distances to the closest cluster center
    inertia.append(kmeans.inertia_)
plt.plot(range(1, clusters, step), inertia)
plt.xlabel('Number of clusters')
```

Out[39]: Text(0.5, 0, 'Number of clusters')



There appears to be an elbow about 50, so we'll use 50 clusters.

```
In [40]: n_clusters = 50
kmeans = KMeans(
    n_clusters=n_clusters,
    max_iter=max_iter,
    n_init=n_init,
    random_state=random_state,
).fit(X_lsa)
```

```
In [41]: # create a new column `title_cluster` and assign it the kmeans cluster labels
# First we need to map the labels to df_unique_articles article ids and then app
article_cluster_map = dict(zip(df_unique_articles['article_id'], kmeans.labels_))
df['title_cluster'] = df['article_id'].map(article_cluster_map)
```

```
In [42]: # Let's check the number of articles in each cluster
np.array(np.unique(kmeans.labels_, return_counts=True)).T
```

```
Out[42]: array([[ 0, 12],
                [ 1, 20],
                [ 2, 23],
                [ 3, 32],
                [ 4, 65],
                [ 5, 38],
                [ 6, 20],
                [ 7, 37],
                [ 8, 18],
                [ 9, 19],
                [10, 11],
                [11, 28],
                [12, 11],
                [13, 11],
                [14,  8],
                [15, 15],
                [16, 29],
                [17, 10],
                [18,  8],
                [19, 13],
                [20, 10],
                [21, 16],
                [22,  8],
                [23, 11],
                [24, 18],
                [25, 20],
                [26,  6],
                [27, 10],
                [28,  5],
                [29,  7],
                [30,  9],
                [31,  7],
                [32,  9],
                [33, 13],
                [34,  7],
                [35,  8],
                [36,  8],
                [37,  7],
                [38,  8],
                [39,  8],
                [40,  9],
                [41,  3],
                [42, 12],
                [43, 12],
                [44,  5],
                [45, 13],
                [46,  8],
                [47,  9],
                [48, 10],
                [49, 10]])
```

```
In [43]: def get_similar_articles(article_id, df=df):
        """
        INPUT:
        article_id - (int) an article id
        df - (pandas dataframe) df as defined at the top of the notebook

        OUTPUT:
        article_ids - (list) a list of article ids that are in the same title cluste
```

```

Description:
Returns a list of the article ids that are in the same title cluster
"""
title_cluster = df[df['article_id'] == article_id]['title_cluster'].values[0]
articles_in_cluster = df[df['title_cluster'] == title_cluster]['article_id']

# remove the input article_id from the list
articles_in_cluster = [a for a in articles_in_cluster if a != article_id]

return articles_in_cluster

```

```

In [44]: def make_content_recs(article_id, n, df=df):
        """
        INPUT:
        article_id - (int) an article id
        n - (int) the number of recommendations you want similar to the article id
        df - (pandas dataframe) df as defined at the top of the notebook

        OUTPUT:
        n_ranked_similar_articles - (list) a list of article ids that are in the same
                                   by popularity
        n_ranked_article_names - (list) a list of article names associated with the

        Description:
        Returns a list of the n most ranked similar articles to a given article_id b
        cluster in df. Rank similar articles using the function get_ranked_article_u
        """
        similar_articles = get_similar_articles(article_id, df=df)
        ranked = get_ranked_article_unique_counts(similar_articles)
        n_ranked_similar_articles = [a[0] for a in ranked[:n]]
        n_ranked_article_names = get_article_names(n_ranked_similar_articles)

        return n_ranked_similar_articles, n_ranked_article_names

```

```

In [45]: # Test out your content recommendations given article_id 25
rec_article_ids, rec_article_titles = make_content_recs(25, 10)
print(rec_article_ids)
print(rec_article_titles)

```

```

[1025, 101, 975, 766, 508, 547, 132, 878, 92, 693]
['data tidying in data science experience', 'how to choose a project to practice
data science', 'the data science process', 'making data science a team sport', 'd
ata science in the cloud', 'trust in data science', 'collecting data science chea
t sheets', '10 data science podcasts you need to be listening to right now', '9 m
istakes to avoid when starting your career in data science', 'better together: sp
ss and data science experience']

```

```

In [46]: assert len({1025, 593, 349, 821, 464, 29, 1042, 693, 524, 352}.intersection(set(

```

2. Now that you have put together your content-based recommendation system, use the cell below to write a summary explaining how your content based recommender works. Do you see any possible improvements that could be made to your function? What other text data would be useful to help make better recommendations besides the article title?

The content-based recommendation system works by:

1. **TF-IDF Vectorization:** Article titles are converted into numerical feature vectors using TF-IDF, which captures the importance of words relative to the corpus.
2. **Dimensionality Reduction:** LSA (TruncatedSVD + normalization) reduces the feature space to 50 components.
3. **KMeans Clustering:** Articles are grouped into 50 clusters based on their reduced feature vectors.
4. **Recommendation:** Given an article, we find all articles in the same cluster and rank them by popularity (number of unique user interactions).

Possible improvements:

- Use article body text instead of just titles for richer content representation.
- Use cosine similarity between TF-IDF vectors directly instead of hard clustering.
- Combine content similarity with collaborative filtering for hybrid recommendations.

Additional useful text data:

- Article descriptions/summaries, tags/categories, author information, and full article body text would all improve content-based recommendations.

Part V: Matrix Factorization

In this part of the notebook, you will build use matrix factorization to make article recommendations to users.

1. You should have already created a **user_item** matrix above in **question 1** of **Part III** above. This first question here will just require that you run the cells to get things set up for the rest of **Part V** of the notebook.

```
In [47]: # quick look at the matrix
user_item.head()
```

```
Out[47]: article_id  0  2  4  8  9 12 14 15 16 18 ... 1434 1435 1436 1437 1439 144
```

	user_id															
1	0	0	0	0	0	0	0	0	0	0	...	0	0	1	0	1
2	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0
3	0	0	0	0	0	1	0	0	0	0	...	0	0	1	0	0
4	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0
5	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0

5 rows × 714 columns

2. In this situation, you can use Singular Value Decomposition from [scikit-learn](#) on the user-item matrix. Use the cell to perform SVD.

```
In [48]: from sklearn.decomposition import TruncatedSVD
from sklearn.metrics import precision_score, recall_score, accuracy_score
```

```
# Using the full number of components which equals the number of columns
svd = TruncatedSVD(n_components=len(user_item.columns), n_iter=5, random_state=4)

u = svd.fit_transform(user_item)
v = svd.components_
s = svd.singular_values_
print('u', u.shape)
print('s', s.shape)
print('vt', v.shape)
```

```
u (5149, 714)
s (714,)
vt (714, 714)
```

3. Now for the tricky part, how do we choose the number of latent features to use? Running the below cell, you can see that as the number of latent features increases, we obtain better metrics when making predictions for the 1 and 0 values in the user-item matrix. Run the cell below to get an idea of how our metrics improve as we increase the number of latent features.

Why SVD works for recommendations:

Singular Value Decomposition (SVD) decomposes the user-item interaction matrix into three matrices: U (user-latent features), Σ (singular values representing feature importance), and V^T (latent features-articles).

The key insight is that SVD discovers **latent factors** — hidden patterns in user behavior and article characteristics that aren't directly observable. For example, a latent factor might capture "interest in deep learning" or "preference for beginner tutorials," even though these categories aren't explicitly labeled in the data.

By reducing the number of latent features (using fewer singular values), we achieve two benefits:

1. **Dimensionality reduction:** The compressed representation is computationally efficient and captures the most important patterns.
2. **Noise filtering:** Discarding smaller singular values removes noise and spurious correlations, leading to better generalization.

The reconstructed matrix ($U \times \Sigma \times V^T$ with reduced dimensions) fills in the "gaps" — predicting interactions that haven't occurred yet but are likely based on latent factor similarity. Articles with similar latent representations to ones a user has already read are strong recommendation candidates.

```
In [49]: num_latent_feats = np.arange(10, 700+10, 20)
metric_scores = []

for k in num_latent_feats:
    # restructure with k latent features
    u_new, vt_new = u[:, :k], v[:k, :]

    # take dot product
    user_item_est = abs(np.around(np.dot(u_new, vt_new))).astype(int)
    # make sure the values are between 0 and 1
```

```

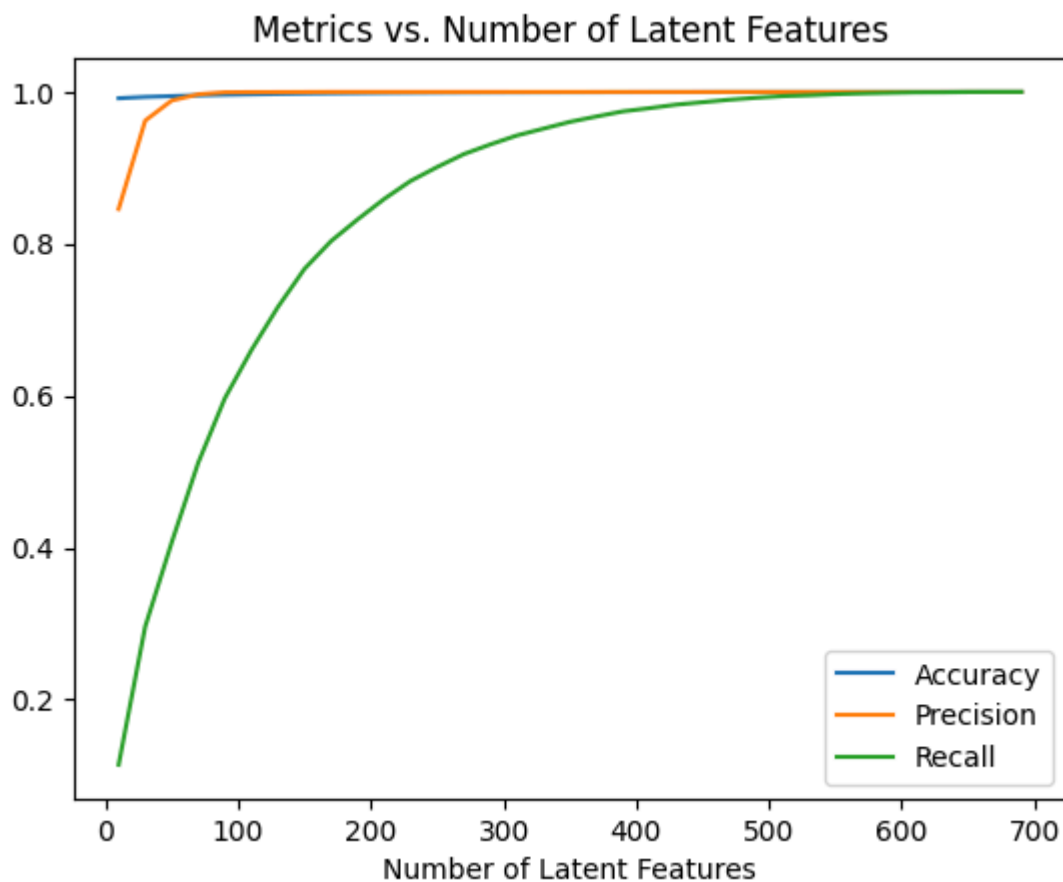
user_item_est = np.clip(user_item_est, 0, 1)

# total errors and keep track of them
acc = accuracy_score(user_item.values.flatten(), user_item_est.flatten())
precision = precision_score(user_item.values.flatten(), user_item_est.flatten())
recall = recall_score(user_item.values.flatten(), user_item_est.flatten())
metric_scores.append([acc, precision, recall])

plt.plot(num_latent_feats, metric_scores, label=['Accuracy', 'Precision', 'Recall'])
plt.legend()
plt.xlabel('Number of Latent Features')
plt.title('Metrics vs. Number of Latent Features')

```

Out[49]: Text(0.5, 1.0, 'Metrics vs. Number of Latent Features')



4. From the above, we can't really be sure how many features to use, because simply having a better way to predict the 1's and 0's of the matrix doesn't exactly give us an indication of if we are able to make good recommendations. Given the plot above, what would you pick for the number of latent features and why?

Based on the plot, I would choose around **200 latent features**. At this point, accuracy, precision, and recall have all reached relatively high levels and begin to plateau. Using more features beyond this point provides diminishing returns while increasing computational cost and the risk of overfitting to noise in the data.

However, it's important to note that optimizing for reconstruction accuracy of the user-item matrix doesn't necessarily translate to better recommendations. The true test would

be an A/B test or holdout evaluation measuring actual user engagement with recommended articles.

5. Using 200 latent features and the values of U, S, and V transpose we calculated above, create an article id recommendation function that finds similar article ids to the one provide.

Create a list of 10 recommendations that are similar to article with id 4. The function should provide these recommendations by finding articles that have the most similar latent features as the provided article.

```
In [50]: def get_svd_similar_article_ids(article_id, vt, user_item=user_item, include_sim
        """
        INPUT:
        article_id - (int) an article id
        vt - (numpy array) vt matrix from SVD
        user_item - (pandas dataframe) matrix of users by articles:
                    1's when a user has interacted with an article, 0 otherwise
        include_similarity - (bool) whether to include the similarity in the output

        OUTPUT:
        article_ids - (list) a list of article ids that are in the same title cluste

        Description:
        Returns a list of the article ids similar using SVD factorization
        """
        # Find the index of the article_id
        article_idx = list(user_item.columns).index(article_id)
        # Find the cosine similarity of all articles
        cos_sim = cosine_similarity(vt.T)
        # Get similarities only for the cos_sim of the article_idx
        sim_scores = cos_sim[article_idx]

        sim_df = pd.DataFrame({
            'article_id': user_item.columns,
            'similarity': sim_scores
        })
        # Sort and return the articles, don't include the own article
        sim_df = sim_df[sim_df['article_id'] != article_id]
        sim_df = sim_df.sort_values('similarity', ascending=False)

        most_similar_items = sim_df['article_id'].tolist()

        if include_similarity:
            return sim_df[['article_id', 'similarity']].values.tolist()
        return most_similar_items
```

```
In [51]: # Create a vt_new matrix with 200 Latent features
        k = 200
        vt_new = v[:,k, :]
```

```
In [52]: # What is the article name for article_id 4?
        print("Current article:", get_article_names([4], df=df)[0])
```

Current article: analyze ny restaurant data using spark in dsx

```

In [53]: # What are the top 10 most similar articles to article_id 4?
rec_articles = get_svd_similar_article_ids(4, vt_new, user_item=user_item)[:10]
rec_articles

Out[53]: [1199, 1068, 486, 1202, 176, 1120, 244, 793, 58, 132]

In [54]: # What are the top 10 most similar articles to article_id 4?
get_article_names(rec_articles, df=df)

Out[54]: ['country statistics: crude oil - exports',
'airbnb data for analytics: athens reviews',
'use spark r to load and analyze data',
'country statistics: crude oil - proved reserves',
'top analytics tools in 2016',
'airbnb data for analytics: paris calendar',
'notebooks: a power tool for data scientists',
'10 powerful features on watson data platform, no coding necessary',
'advancements in the spark community',
'collecting data science cheat sheets']

In [55]: assert set(rec_articles) == {1199, 1068, 486, 1202, 176, 1120, 244, 793, 58, 132}
print("That's right! Great job!")

```

That's right! Great job!

6. Use the cell below to comment on the results you found in the previous question. Given the circumstances of your results, discuss what you might do to determine if the recommendations you make above are an improvement to how users currently find articles, either by Sections 2, 3, or 4? Add any tradeoffs between each of the methods, and how you could leverage each type for different situations including new users with no history, recently new users with little history, and users with a lot of history.

Results Discussion:

The SVD-based recommendations for article 4 show articles with similar latent feature patterns, which often capture underlying topics and user behavior patterns.

Testing Recommendations: To determine if these recommendations improve user engagement, we could:

1. **A/B Testing:** Show SVD recommendations to one group and current recommendations to another, measuring click-through rates and time spent.
2. **Holdout Evaluation:** Split data temporally — train on earlier interactions, test on later ones, and measure precision/recall.

Method Tradeoffs:

- **Rank-based (Part II):** Simple, works for all users including new ones, but not personalized. Best for cold-start/new users.
- **Collaborative filtering (Part III):** Personalized based on similar users' behavior, but suffers from cold-start problem for new users.
- **Content-based (Part IV):** Can recommend based on article similarity without user history, useful for new users with known interests.

- **Matrix Factorization (Part V):** Captures latent patterns, can handle sparse data well, but requires sufficient interaction history. Best for users with moderate to extensive history.

Strategy by user type:

- **New users (no history):** Use rank-based or content-based recommendations.
- **Recent users (little history):** Blend rank-based with early collaborative filtering signals.
- **Established users (lots of history):** Use collaborative filtering or SVD-based recommendations for maximum personalization.

Extras

Using your workbook, you could now save your recommendations for each user, develop a class to make new predictions and update your results, and make a flask app to deploy your results. These tasks are beyond what is required for this project. However, from what you learned in the lessons, you certainly capable of taking these tasks on to improve upon your work here!

Conclusion

Congratulations! You have reached the end of the Recommendation Systems project!

Tip: Once you are satisfied with your work here, check over your report to make sure that it satisfies all the areas of the [rubric](#). You should also probably remove all of the "Tips" like this one so that the presentation is as polished as possible.

Directions to Submit

Before you submit your project, you need to create a .html or .pdf version of this notebook in the workspace here. To do that, run the code cell below. If it worked correctly, you should get a return code of 0, and you should see the generated .html file in the workspace directory (click on the orange Jupyter icon in the upper left).

Alternatively, you can download this report as .html via the **File > Download as** submenu, and then manually upload it into the workspace directory by clicking on the orange Jupyter icon in the upper left, then using the Upload button.

Once you've done this, you can submit your project by clicking on the "Submit Project" button in the lower right here. This will create and submit

a zip file with this .ipynb doc and the .html or .pdf version you created.
Congratulations!

```
In [ ]: from subprocess import call  
        call(['python', '-m', 'nbconvert', 'Recommendations_with_IBM.ipynb'])
```

Out[]: 1